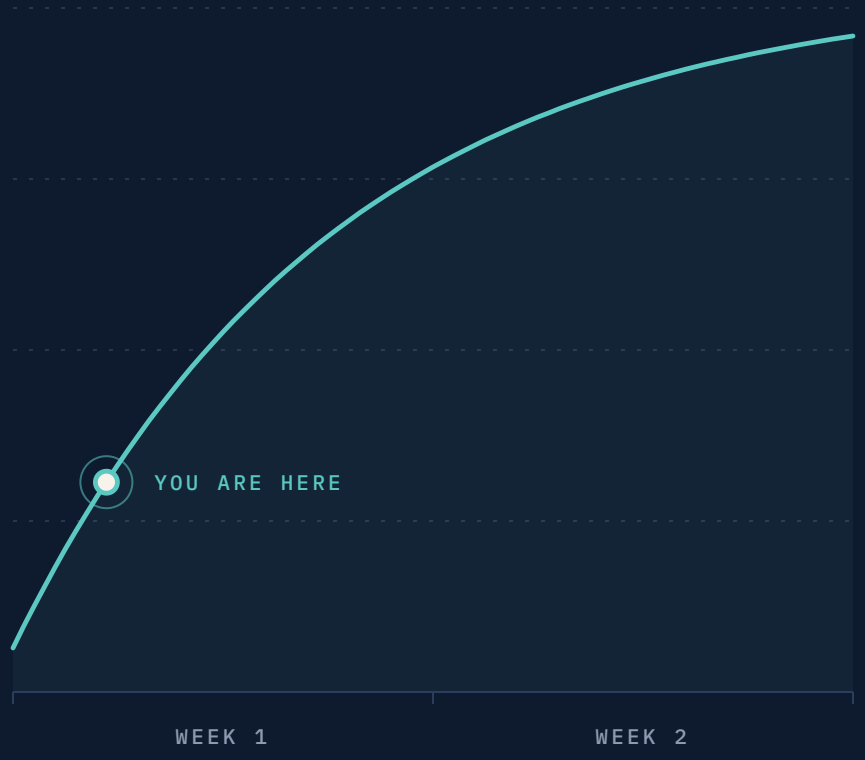
WEEK 1 · DAY 2 · OVERVIEW

Frequency, Lags, and the Naive Baselines



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

A rates and credit desk will try nine days of time series models on five daily series: the 2 Yr and 10 Yr Treasury yields, the 2s10s slope, and an IG and an HY spread.

Before any model, it sets the bar: the score of a rule with no model in it.

Each rule gives the change in a series on a date, in bp, from the dates before it, and is scored against the change that happened.

Textbooks name these forecasting models, but in this course every output is a value for a date inside the file and never a forecast of rates or spreads.

OBJECTIVES

Set the development and held-out years before looking at a value, build lags and changes, and prove a feature uses only earlier dates.

Build three rules with no model in them, score them with MAE, RMSE and MASE, write the walk-forward loop over five folds, and build the baseline table for five series.

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 5 EXERCISES

Your Route Through Day 2

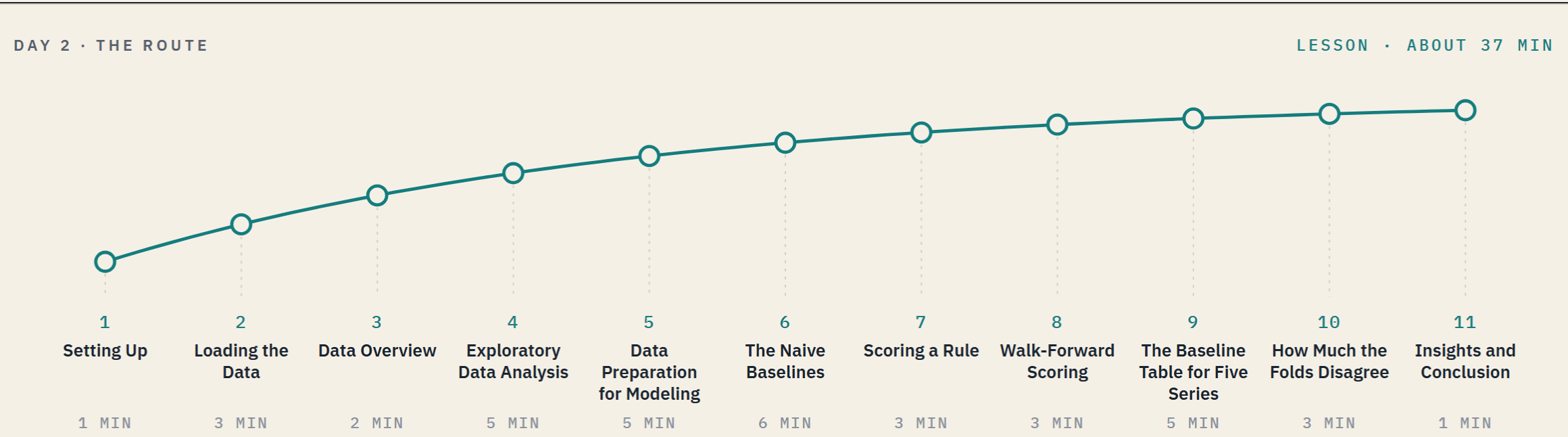


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 37 minutes for the lesson, then about 15 for the five exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ipywidgets import interact
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

Read both files and fix the cut first.

Loading the Data

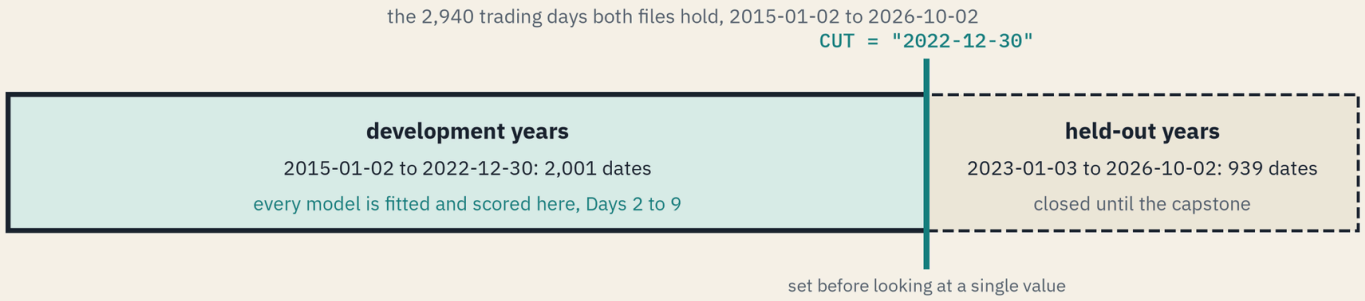


FIGURE 2 The 2,940 trading days cut on 2022-12-30: 2,001 development dates, and 939 held-out dates closed until the capstone.

BOTH FILES

```
(2940, 14) (2940, 2) True
2015-01-02 to 2026-10-02
```

THE CUT

```
CUT = "2022-12-30"
development = yields.loc[:CUT]

Development: 2001 dates, 2015-01-02 to 2022-12-30
Held out: 939 dates, 2023-01-03 to 2026-10-02
```

We fix the cut before looking at a single value; the held-out years stay closed until the capstone.

Data Overview

THE FIVE SERIES IN BP

	2 YR	10 YR	2S10S	IG	HY
count	2001.0000	2001.0000	2001.0000	2001.0000	2001.0000
mean	137.9265	207.8826	69.9560	87.1356	240.5106
std	106.0845	73.8736	53.4732	8.9974	33.1905
min	9.0000	52.0000	-84.0000	64.7000	157.5000
25%	60.0000	160.0000	26.0000	81.3000	215.6000

EVERY WEEKDAY IN THE FILE?

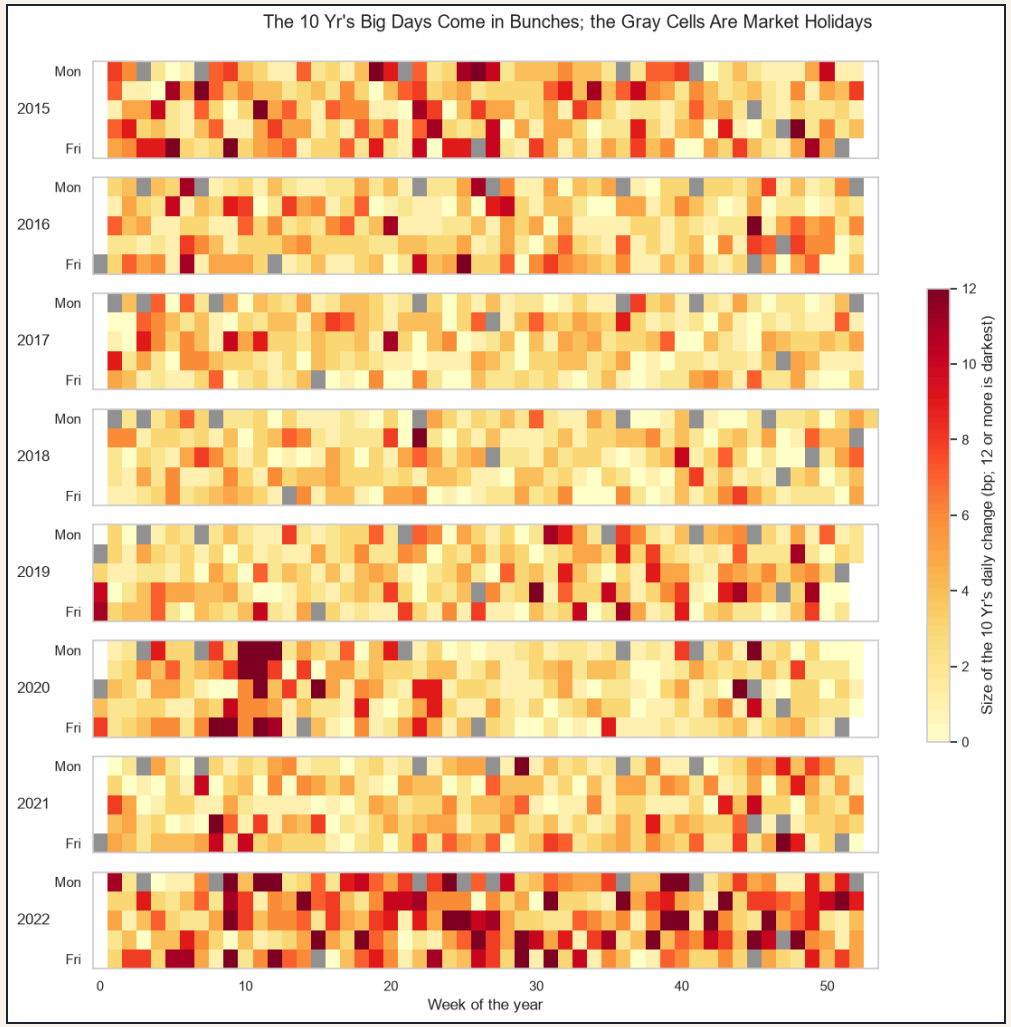
```
Frequency: None
2086 weekdays, 85 with no row, 51 of them Mondays
```

BLANK CELLS

```
Blank cells: 0
```

Five complete series on 2,001 trading days; a weekday with no row is a holiday, not a gap.

Exploratory Data Analysis



THE TEN LARGEST DAYS

```
date
2022-11-10   -30.0000
2020-03-17    29.0000
2022-06-13    28.0000
2022-09-28   -25.0000
2020-03-10    22.0000
2020-03-16   -21.0000
2020-03-09   -20.0000
2020-03-20   -20.0000
2016-11-09    19.0000
2022-09-22    19.0000
Name: 10 Yr, dtype: float64
```

READING THE COLOURS

The colour comes in seasons: an average day moved 6.30 bp in 2022 and 2.73 in 2018.

Data Preparation for Modeling

LAGS AND CHANGES

DATE	LEVEL	LAG 1	CHANGE
2015-01-02	212.0000	NaN	NaN
2015-01-05	204.0000	212.0000	-8.0000
2015-01-06	197.0000	204.0000	-7.0000
2015-01-07	196.0000	197.0000	-1.0000
2015-01-08	203.0000	196.0000	7.0000

THE BLANKED COPY

```
blanked = ten_changes.copy()
blanked.loc[TEST_CUT:] = np.nan
```

THE QUESTION

Q. Under the cut test, how many of its 1,249 values up to the cut move? Write down a number.

THE CUT TEST

```
def cut_test(feature):
    whole, stopped = feature(ten_changes).loc[:TEST_CUT], feature(blanked).loc[:TEST_CUT]
    print("equal:", whole.equals(stopped), "|", len(whole.compare(stopped)), "of", len(whole), "values moved")
```

Every lag and rolling value a model reads in this course passes the cut test first.

The Naive Baselines

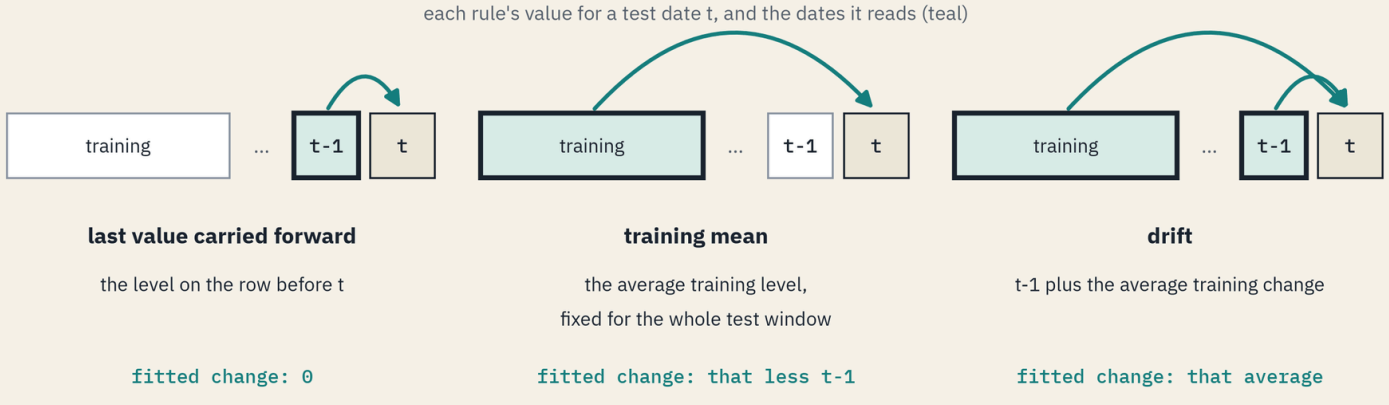


FIGURE 4 Each rule reads only dates before t: the last value the row before, the training mean and drift the training dates.

THE THREE RULES

The last value carried forward: a date's value is the level on the row before.

The training mean: every test date gets the average level of the training dates.

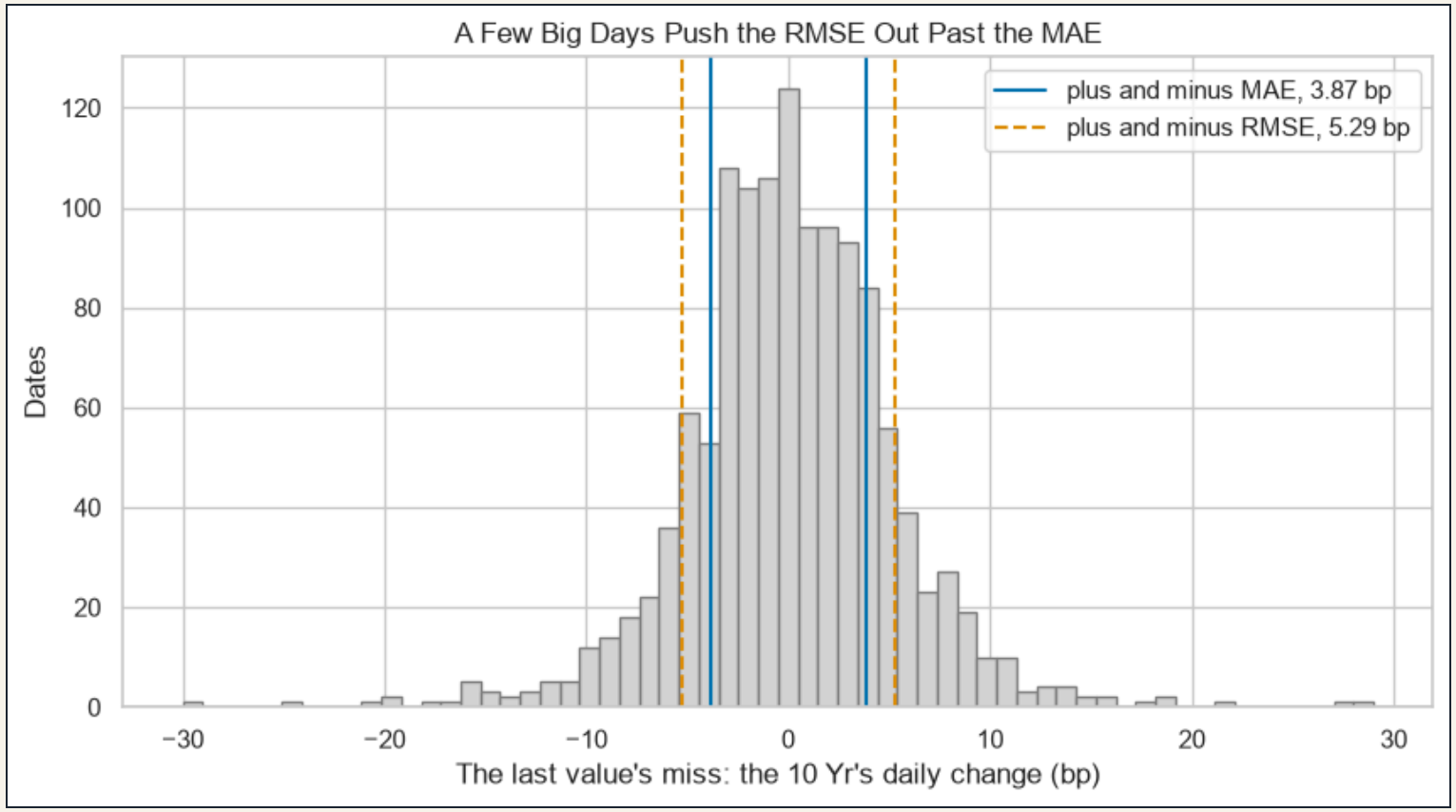
Drift: the row before plus the average training change.

THE FOLDS

```
Training: 740 changes, 2015-01-05 to 2017-12-14
Test: 252 dates, 2017-12-15 to 2018-12-19
```

A naive baseline is a rule with no model in it, the score anything fitted must beat.

Scoring a Rule



TWO SCORES

- **MAE**: the average size of a miss.
- **RMSE**: the root of the average squared miss, so a big miss weighs more.

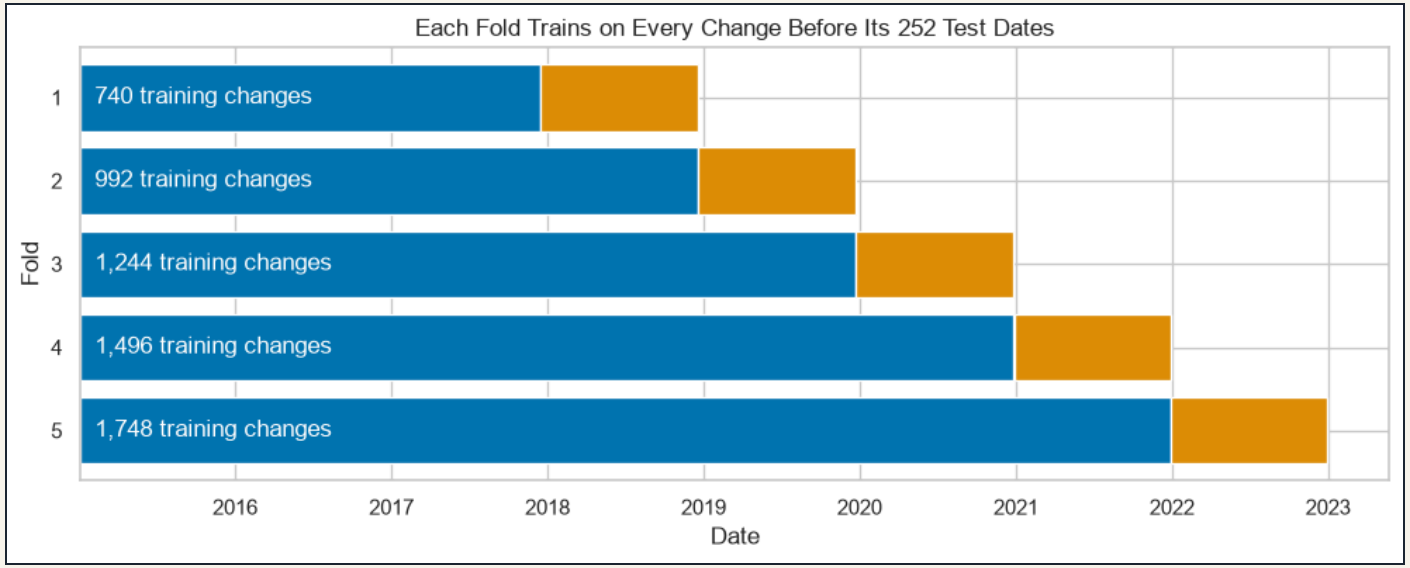
THE TEN LARGEST DAYS

```
Share of squared misses: 15.8 percent
Share of absolute misses: 4.8 percent
```

Every model in this course ends in a MASE against the last value.

Draw the five folds and write the loop.

Walk-Forward Scoring



THE LOOP

```
for fold, (fit_rows, test_rows) in enumerate(folds, start=1):
    drift = last_value + changes[series][fit_dates].mean()
    training_mean = levels[series][fit_dates].mean() - levels[series].shift(1)[test_dates]
```

Each rule is fitted once a fold, on the dates before it, and scored date by date.

The Baseline Table for Five Series

THE LOOP OVER FIVE SERIES

```
rows = []
for series in levels.columns:
    for fold, (fit_rows, test_rows) in enumerate(folds, start=1):
        fit_dates, test_dates = changes.index[fit_rows], changes.index[test_rows]
        actual = changes[series][test_dates]
        last_value = 0 * actual
        drift = last_value + changes[series][fit_dates].mean()
        training_mean = levels[series][fit_dates].mean() - levels[series].shift(1)[test_dates]
        rows.append([series, fold, mean_absolute_error(actual, last_value), mean_absolute_error(actual, drift),
                     mean_absolute_error(actual, training_mean)])
fold_scores = pd.DataFrame(rows, columns=["series", "fold", "last value", "drift", "training mean"])
print("Drift lower in", (fold_scores["drift"] < fold_scores["last value"]).sum(), "of", len(fold_scores))
fold_scores[fold_scores["drift"] < fold_scores["last value"]]
```

THE QUESTION

Q. Across five series and five folds, in how many of the 25 does drift have the lower MAE? Write down a number.

How Much the Folds Disagree

THE LAST VALUE'S FOLD RANGE, IN BP

SERIES	MIN	MAX	FOLD RANGE
2 Yr	1.4881	6.0675	4.5794
10 Yr	2.6905	6.2579	3.5675
2s10s	1.6468	3.8413	2.1944
IG	0.7349	1.2087	0.4738
HY	3.0635	5.0329	1.9694

THE QUESTION

Q. What is the average of the last value's 20 fold MAEs on the 10 Yr? Write down a number in bp.

TWENTY SHORTER FOLDS

```
short_folds = TimeSeriesSplit(n_splits=20, test_size=63).split(changes)
short = [changes["10 Yr"].iloc[test_rows].abs().mean() for fit_rows, test_rows in short_folds]
print("Average:", round(np.mean(short), 3), "bp | folds:", round(min(short), 2), "to", round(max(short), 2), "bp")
```

A model that beats the last value by less than the folds disagree has not beaten it.

Insights and Conclusion



DAILY CHANGES

```
(2000, 5) from 2015-01-05
```

WHERE EACH STARTS AND ENDS

The 10 Yr opens at 212 bp and closes at 388; the 2s10s slope opens at 146 and closes at -53.

On a daily series, beat the last value by more than the folds disagree, or keep the last value.

You Can Now

- Set development and held-out years before looking at the data.
- Build lags and changes by rows, and prove with the cut test that a feature uses only earlier dates.
- Build the last value carried forward, the training mean and drift, and score each with MAE, RMSE and MASE.
- Write the walk-forward loop over **`TimeSeriesSplit`** folds and build a baseline table for several series.
- Read how far the folds disagree before calling one rule better than another.